PanoMito User Manual

A comprehensive guide for installation, data preparation, training, inference, benchmarking, clustering, and visualization.

Version aligned with repository - 2026-06-30

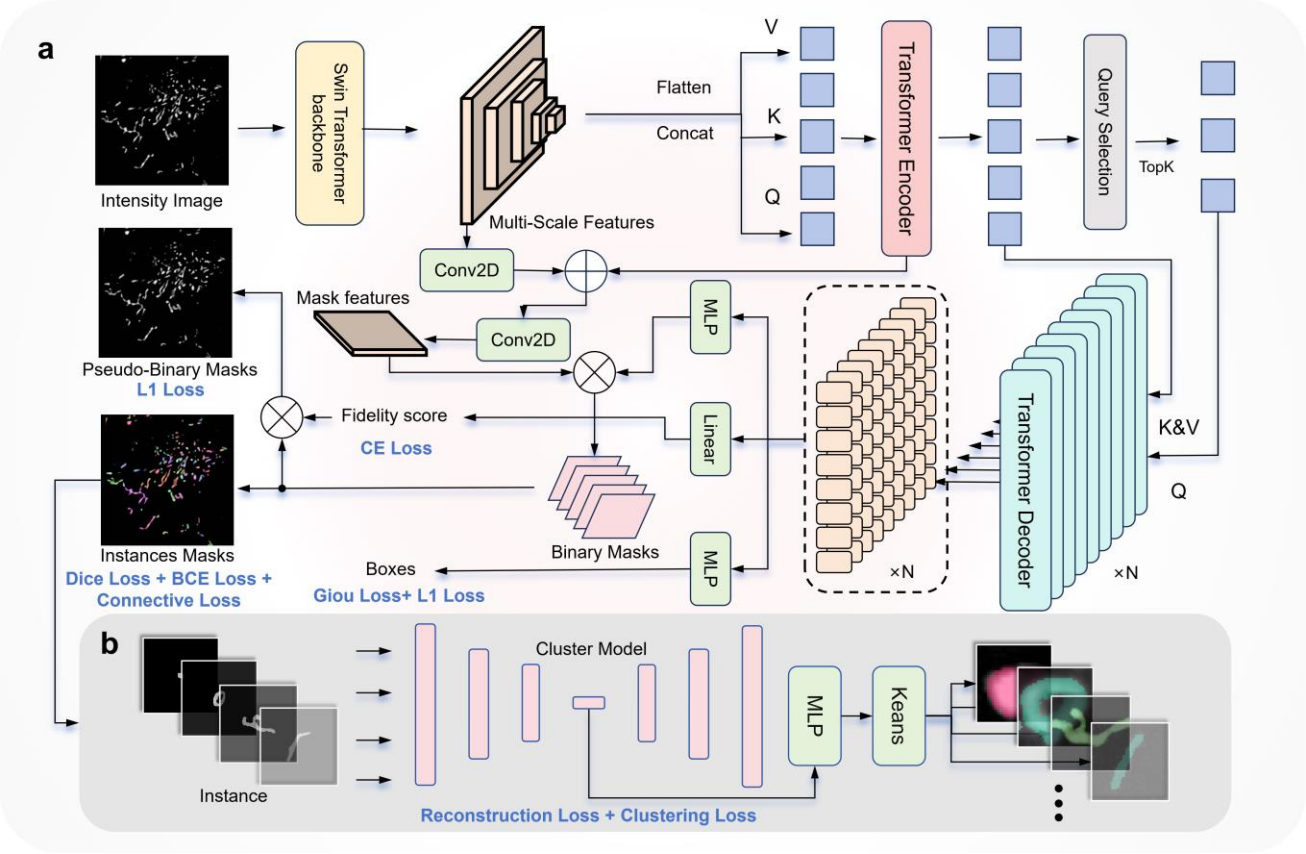

Table of Contents

1. Introduction

PanoMito is a comprehensive toolbox for mitochondrial analysis. It includes the PanoMitoAtlas dataset (130,000+ annotated instances), the PanoMitoSeg segmentation model, the PanoMitoCluster clustering module, and integrated analysis pipelines. The toolbox supports microscopy images of arbitrary sizes via tiled inference and is designed for both reproducing published experiments and adapting to custom datasets.

Website: http://www.panomito.com/ | Dataset Zenodo: https://zenodo.org/records/21129338 | Models Zenodo: https://zenodo.org/records/20959653

2. System Requirements and Installation

2.1 System requirements

- OS: Ubuntu 20.04 or 22.04 (tested).
- GPU: NVIDIA GPU with CUDA; RTX 3090 (24 GB) and RTX 5090 (32 GB) tested.
- VRAM: minimum 16 GB recommended for 512x512 tiled inference.
- Python: 3.9.21 (conda environment PanoMito_demo).
- PyTorch/CUDA: choose the installation command matching your GPU and driver; see tested cases in Section 2.3.

2.2 External repositories

```
git clone git@github.com:facebookresearch/detectron2.git
git clone git@github.com:fundamentalvision/Deformable-DETR.git
```

2.3 Environment setup

2.3.1 Create and activate the conda environment:

```
conda create --name PanoMito_demo python=3.9.21
conda activate PanoMito_demo
```

2.3.2 Install PyTorch:

Choose the installation command that matches your GPU and driver. The following two cases have been tested on our machines:

	Case 1	Case 2
OS	Ubuntu 20.04	Ubuntu 22.04
GPU	NVIDIA GeForce RTX 3090 (24 GB)	NVIDIA GeForce RTX 5090 (32 GB)
Driver	535.230.02	575.64.05
CUDA (driver)	12.2	12.9

Case 1 (RTX 3090):

```
conda install pytorch=1.12.0 torchvision=0.13.0 torchaudio=0.12.0 cudatoolkit=11.3 -c pytorch
```

Case 2 (RTX 5090):

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu128
conda install -c "nvidia/label/cuda-12.8.0" cuda-toolkit
```

Other hardware: If your GPU model, driver, or CUDA version differs, visit the PyTorch official website (https://pytorch.org/get-started/locally/) to select the matching install command for your system.

2.3.3 Install remaining dependencies and compile external operators:

Before compiling, make sure **nvcc** is installed and on your PATH (verify with nvcc --version). Also confirm that your **GCC** version is compatible with your CUDA toolkit—mismatched versions are a common cause of build failures. See the CUDA–GCC compatibility table for supported combinations.

```
pip install -r requirements.txt

cd detectron2

pip install --no-build-isolation --config-settings editable_mode=compat -e .

cd ../Deformable-DETR/models/ops

sh ./make.sh
```

LD_LIBRARY_PATH note: If CUDA errors such as CUBLAS_STATUS_INVALID_VALUE occur due to mixed system/conda libraries, unset LD_LIBRARY_PATH before running Python, or use the re-exec guard already implemented in train.py and predict.py.

2.4 Python dependencies

cython, scipy, shapely, timm, h5py, submitit, scikit-image, opencv-python, pycocotools, gdown, numpy, sahi, scikit-learn, pandas, umap-learn, anndata. Runtime also requires torch, detectron2, tqdm, tifffile, matplotlib, PIL.

3. Repository Structure

Path	Description
train.py	Fine-tune PanoMitoSeg
predict.py	Tiled segmentation inference
data_pre.py	COCO JSON -> RLE .npy conversion
configs/	MaskDINO YAML configuration files
panomito/	Core Python package (model, utils, cluster)
maskdino/	MaskDINO model implementation
demo/	Benchmark and clustering demo scripts
cellpose_benchmark/	Cellpose baseline comparison
data/train/, data/test/	Training and test images + annotations
MODEL/	PanoMitoSeg.pth, PanoMitoCluster.pth, cellpose_model
pretrain_model/	TissueNet initialization weights
results/	Inference and demo outputs
output/train/	Training checkpoints and logs
figures/	Documentation figures

4. Resources and Data Preparation

4.1 PanoMitoAtlas dataset

Download PanoMitoAtlas.zip from https://zenodo.org/records/21129338. The bundle includes 2D, time-lapse images and train/test splits used in the paper.

If you want to reproduce paper results:

- Extract PanoMitoAtlas.zip.
- Copy train_test_subset/ contents into ./data/ (train -> data/train/, test -> data/test/).
- Ensure PNG images and JSON annotations are in the same folders.

4.2 Model files

File	Purpose
PanoMitoSeg.pth	Segmentation model for inference and demos
PanoMitoCluster.pth	Clustering autoencoder weights
cellpose_model	Cellpose baseline (cellpose_benchmark/)

Download the three models listed above from https://zenodo.org/records/20959653 and place them in "MODEL" folder.

4.3 Pretrain weights

File	Purpose
tissuenet_model_0019999.pth	TissueNet-pretrained MaskDINO weights used as initialization for "train.py"

Download the "tissuenet_model_0019999.pth" from https://zenodo.org/records/20959653 and place them in "pretrain_model" folder.

4.4 Data preparation

A. Paper pre-split (default)

- Download PanoMitoAtlas from https://zenodo.org/records/21129338 and extract it.
- Copy the images and the corresponding COCO-format annotation file from "*train_test_subset/train/*" into "*data/train/*".
- Copy the images and the corresponding COCO-format annotation file from "*train_test_subset/test/*" into "*data/test/*".
- Run: python data_pre.py
- Outputs: "*_train_subdataset_gt_nocate_rle.npy*" and "*_test_subdataset_gt_nocate_rle.npy*"

B. Custom train/test split

- Use annotations from "*all_2d_dataset/*" or your own merged COCO-format JSON file.
- Before calling "*coco2npy_rle()*" in "*data_pre.py*", follow the comments in "*data_pre.py*" to call "*split_coco_dataset_traintest()"* and generate custom train/test JSON files.
- Modify the data-reading paths and annotation paths according to your dataset organization to ensure that the script can correctly locate the image files and annotation JSON files.

5. Core Workflow: Training (train.py)

Fine-tune PanoMitoSeg on annotated training data. train.py reads RLE-encoded .npy annotations, while the raw release annotations are provided in COCO JSON format. Use data_pre.py to convert COCO JSON annotations before training.

Step 1 - Prepare images and JSON under ./data/

Follow Section 4.4 to download PanoMitoAtlas and prepare the data.

Step 2 - Start fine-tuning

```
python train.py
```

Input

- Dataset: RLE-encoded .npy files under ./data/train/ and ./data/test/.
- Pretrained weights: ./pretrain_model/tissuenet_model_0019999.pth.

Output (under ./output/train/)

- config.yaml - snapshot of the full training configuration used for this run.
- log.txt - text log with iteration losses, learning rate, evaluation AP/AR, and timestamps.
- metrics.json - training metrics in JSON format, useful for plotting training curves.
- events.out.tfevents.* - TensorBoard event files; view with tensorboard --logdir output/train.
- model_0004999.pth, model_0009999.pth, ... - periodic checkpoints saved during training.
- model_final.pth - final model weights at the end of training.
- last_checkpoint - pointer file to the most recently saved checkpoint, used for resume.
- inference/coco_instances_results.json - COCO-format predictions on the test set used to compute AP/AR during training.

Evaluation metrics are printed to the terminal and recorded in log.txt at each evaluation step. Additional Detectron2 CLI arguments are supported. If needed, edit GPU settings and training overrides in train.py before running.

6. Core Workflow: Prediction (predict.py)

Run tiled segmentation inference on microscopy images using the trained PanoMitoSeg model. The workflow supports images of arbitrary size by splitting each image into overlapping tiles and merging predicted instances across tiles.

Command

```
python predict.py
```

Input

- Images: ./data/test/ (PNG images by default).
- Model: ./MODEL/PanoMitoSeg.pth.

Pipeline

- Load PanoMitoSeg.pth via PanomitoPredictor.
- Split each PNG image into 512 x 512 tiles with 50% overlap.
- Run inference per tile and merge instances across tiles.
- Apply optional NMS post-processing (postprocess='use_NMS').
- Export COCO-format instance segmentation results.

Output

- ./results/predict_results/_sub_predictor_use_NMS_dataset=all_merge_thresh=0.4_c=0.4_merge_q300.json - COCO-format instance segmentation results.

Default inference settings include tile size 512, merge_thresh=0.4, output_min_score=0.4, merge_iou=0.8, containment_threshold=0.8, and NMS post-processing. Edit the arguments in predict.py or call run_panomito_predict() to customize paths and thresholds.

7. Demo: Benchmark Evaluation

The benchmark demo runs the same prediction logic as predict.py, then aligns prediction IDs to ground truth, computes AP/AR/F1 metrics, and saves representative visualizations. This demo is intended for reproducing the segmentation benchmark using the predefined test set.

Command

```
python demo/panomito_benchmark.py
```

Input

- Images: ./data/test/ (PNG).
- Ground truth: ./data/test/_test_subdataset_gt.json.
- Model: ./MODEL/PanoMitoSeg.pth.

Output (under ./results/panomito_benchmark/)

- _sub_predictor_use_NMS_dataset=all_merge_thresh=0.4_c=0.0_merge_q300.json - COCO-format predictions.
- _sub_predictor_use_NMS_dataset=all_merge_thresh=0.4_c=0.0_merge_q300_adjust_id.json - predictions with IDs aligned to ground truth.
- Evaluation metrics printed to the terminal.
- Sample visualizations in visualization/.
- Timestamped log: panomito_benchmark_YYYYMMDD_HHMMSS.txt.

8. Demo: Clustering Analysis

The clustering demo performs segmentation followed by morphological clustering and morphology class assignment. It is an end-to-end example of using PanoMitoSeg together with PanoMitoCluster.

Pipeline steps: segmentation -> instance mask export -> K-means clustering (K=12) -> morphology classification summary.

Command

```
python demo/predict_cluster_analysis.py
```

Input

- Images: ./data/test/ (PNG).
- Models: ./MODEL/PanoMitoSeg.pth and ./MODEL/PanoMitoCluster.pth.

Output (under ./results/predict_cluster_analysis/)

Root directory

- _sub_predictor_use_NMS_dataset=all_merge_thresh=0.4_c=0.0_merge_q300.json - COCO-format segmentation results.
- predict_cluster_analysis_instance_metrics.csv - per-instance morphology metrics with morphology class labels.
- predict_cluster_analysis_cluster_summary.csv - per-cluster mean metrics and morphology class assignment; this is the final morphology class result.

MitoInstance/

- {annotation_id:06d}_all_data.png - per-instance 255 x 255 binary mask crops.
- all_z_np.npy - autoencoder latent features used for clustering.

Cluster/

- mito_clusters.csv - filename-to-cluster mapping with K-means labels 0-11.
- clustering_umap.png - 2D UMAP visualization of cluster embeddings.
- KmeansLabelRefine_0/ ... KmeansLabelRefine_11/ - intermediate clustering outputs: instance masks grouped by K-means cluster, used to extract per-cluster metrics and as input for morphology classification.

9. CellPose Baseline Benchmark

This optional workflow reproduces the CellPose baseline comparison using the custom CellPose model provided with the model resources.

Command

```
python cellpose_benchmark/cellpose_benchmark.py
```

Requirements

```
pip install -r cellpose_benchmark/cellpose_requirements.txt
```

Item	Path / value
Input images	./data/test/*.png
Model	./MODEL/cellpose_model
Ground truth	./data/test/_test_subdataset_gt.json
Output directory	./results/cellpose_pre/
diameter	25
flow_threshold	0.4

The CellPose predictions and evaluation outputs are saved under ./results/cellpose_pre/.

10. CVAT Visualization

PanoMito prediction results are saved in COCO-style JSON format and can be visualized interactively using CVAT. This is useful for inspecting predicted mitochondrial instance masks, comparing predictions with manual annotations, and reviewing segmentation quality on custom datasets.

1. Prepare images and annotation files

Before visualization, make sure that the image files and the COCO-format JSON file use consistent file names. For example, if the prediction JSON contains an image entry named sample_001.png, the corresponding image file uploaded to CVAT should have the same name.

File	Description
./data/test/*.png	Input microscopy images
./results/predict_results/*_merge_q300.json	PanoMitoSeg prediction results in COCO format

2. Create a CVAT project

A CVAT project defines the shared label schema for related tasks. Create the project first so labels stay consistent when importing PanoMito predictions later.

- Open CVAT and go to the Projects page.
- Create a new project and enter a project name, such as PanoMito.
- Under Labels, add a label named Mito, using the same label name as in the COCO prediction/annotation JSON.
- Submit the project.

3. Create a CVAT task

A CVAT task holds the microscopy images to visualize. Link it to the project created above so it inherits the Mito label automatically.

- From the project page, create a new task.
- Enter a task name and select the project created above.
- Upload the corresponding microscopy images from your computer. The file names must match those in the COCO JSON.
- Submit the task and open the annotation interface.

4. Import prediction or annotation JSON

- Open the task in CVAT.
- Click Actions -> Upload annotations.
- Select COCO 1.0 as the annotation format.
- Upload the PanoMitoSeg prediction JSON or the ground-truth COCO JSON.
- Refresh the task if masks do not appear immediately.

The predicted mitochondrial instances should then appear as mask annotations overlaid on the original microscopy images.

5. Recommended use cases

- Inspect PanoMitoSeg instance masks on custom images.

- Compare predicted masks with manual annotations.
- Identify merged, fragmented, missed, or duplicate mitochondrial instances.
- Prepare corrected annotations for additional fine-tuning.
- Review difficult cases such as densely packed or intertwined mitochondrial networks.

11. Troubleshooting

Issue	Suggested fix
CUBLAS_STATUS_INVALID_VALUE	Unset LD_LIBRARY_PATH before python; avoid mixing system CUDA and conda libs.
detectron2 pip install fails	Use: pip install --no-build-isolation --config-settings editable_mode=compat -e .
Deformable-DETR compile error	Ensure CUDA toolkit matches PyTorch; run make.sh from models/ops.
train.py missing .npy	Run data_pre.py after placing COCO JSON + PNG under data/train and data/test.
Empty prediction JSON	Check MODEL/PanoMitoSeg.pth exists; verify PNG paths; lower output_min_score.
CVAT import mismatch	Match file_name in JSON exactly to uploaded image names.
Markdown preview image missing	Ensure figures/ is not gitignored; use relative paths.

12. Quick Command Reference

```
python data_pre.py
python train.py
python predict.py
python demo/panomito_benchmark.py
python demo/predict_cluster_analysis.py
python cellpose_benchmark/cellpose_benchmark.py
```

For the latest documentation, see README.md, demo/README.md, and cellpose_benchmark/README.md in the repository.